OAVS TGT Computer Science

MCQ Question Bank

Topic: Character Strings

- **Total Questions:** 140
- **Coverage:** Strings in C (definition, representation, null character, declaration, initialization, storage, reading & displaying, characteristics, applications), Standard Library Functions – strlen(), strcat(), strcpy(), strcmp(), and Similarity between Strings & One-Dimensional Character Arrays
- **Question Types:** Definition-based, Conceptual, Function-based, Syntax-based, Code-snippet, Output-based, Statement-based, Assertion–Reason, Match-the-Following, Comparison-based, Fill-in-the-Blank, Scenario-based, Application-oriented, Previous-Year Pattern, Difficult & Tricky MCQs
- **Format:** Every question includes 4 options, the correct answer, and a revision-oriented explanation

Table of Contents

Chapter 1: Strings in C - Fundamentals (35 Qs) 3

Chapter 2: The strlen() Function (20 Qs) 12

Chapter 3: The strcat() Function (20 Qs) 17

Chapter 4: The strcpy() Function (20 Qs) 22

Chapter 5: The strcmp() Function (20 Qs) 27

Chapter 6: Strings vs One-Dimensional Character Arrays (25 Qs) 33

Chapter 1: Strings in C - Fundamentals

35 Questions | OAVS TGT Computer Science – Character Strings

Q1 | DEFINITION | CHARACTER STRINGS STR

What is a string in C?

A. An array of characters terminated by a null character

B. A collection of integers

C. A pointer to memory

D. A single character

Ans: A

Explanation: In C, a string is an array of characters terminated by the null character '\0'.

Q2 | DEFINITION | CHARACTER STRINGS STR

Which character marks the end of a string in C?

A. '\n'

B. EOF

C. '\0'

D. '\t'

Ans: C

Explanation: The null character '\0' marks the logical end of every C string.

Q3 | CONCEPTUAL | CHARACTER STRINGS STR

What is the ASCII value of the null character '\0'?

A. -1

B. 10

C. 0

D. 1

Ans: C

Explanation: The null character has ASCII value 0, distinguishing it from printable characters.

Q4 | SYNTAX | CHARACTER STRINGS STR

Which of the following is a valid string declaration in C?

A. char str[10];

B. str char[10];

C. char str = "text";

D. string str;

Ans: A

Explanation: Strings are declared as character arrays using char arrayName[size];

Q5 | REPRESENTATION | CHARACTER STRINGS STR

How is a string represented internally in memory?

A. As a stack of characters

B. As a hash table

C. As a contiguous array of characters ending with '\0'

D. As a linked list of characters

Ans: C

Explanation: C stores string characters in contiguous memory cells, followed by a terminating '\0'.

Q6 | INITIALIZATION | CHARACTER STRINGS STR

Which statement correctly and safely initializes the string "Hello"?

A. char str[6] = "Hello";

B. char str = "Hello";

C. char str[5] = "Hello";

D. int str[] = "Hello";

Ans: A

Explanation: "Hello" needs 5 characters plus a null terminator, so the array size must be at least 6.

Q7 | STORAGE | CHARACTER STRINGS STR

What is the minimum array size required to store the string "OAVS" including its null terminator?

A. 3

B. 6

C. 4

D. 5

Ans: D

Explanation: "OAVS" has 4 characters; one extra byte is needed for '\0', giving a minimum size of 5.

Q8 | READING STRINGS | CHARACTER STRINGS STR

Which function/approach is commonly used to read a string containing spaces from user input?

A. getchar() only

B. gets() or fgets()

C. scanf("%d", &x);

D. printf();

Ans: B

Explanation: gets() (unsafe/deprecated) and fgets() can read strings with embedded spaces, unlike scanf("%s").

Q9 | READING STRINGS | CHARACTER STRINGS **STR**

Which format specifier is used with scanf/printf to read or display a string?

A. %d
B. %s
C. %f
D. %c

Ans: B

Explanation: %s is the format specifier reserved for string input/output in C.

Q10 | READING STRINGS | CHARACTER STRINGS **STR**

Why is gets() considered unsafe in modern C programming?

A. It cannot read strings at all
B. It requires <math.h>
C. It runs too slowly
D. It does not check array bounds, causing buffer overflow

Ans: D

Explanation: gets() has no way to limit how many characters it reads, so it can overflow the destination array.

Q11 | CHARACTERISTICS | CHARACTER STRINGS **STR**

Which of the following is a characteristic of strings in C?

A. Strings can only be numeric
B. Strings are terminated by a null character
C. Strings are always fixed at length 10
D. Strings cannot be stored in arrays

Ans: B

Explanation: Null termination is a defining characteristic that lets library functions know where a string ends.

Q12 | STORAGE | CHARACTER STRINGS **STR**

Given char str[10] = "Hello"; what value is stored at str[5]?

A. 'o'
B. '\0'
C. Garbage value
D. 'H'

Ans: B

Explanation: Indices 0-4 hold H-e-l-l-o; index 5 holds the automatically-added null terminator.

Q13 | APPLICATIONS | CHARACTER STRINGS STR

Which of the following is a common real-world application of strings?

A. Matrix multiplication
B. Binary search on numeric arrays
C. Text processing and handling file names
D. Sorting only integers

Ans: C

Explanation: Strings are fundamental to handling text, names, messages, and file paths in programs.

Q14 | INITIALIZATION | CHARACTER STRINGS STR

What happens when a string literal is assigned without specifying array size, e.g. char str[] = "OAVS";?

A. Compiler error
B. Compiler automatically allocates size = length + 1
C. It allocates size = length only, with no null character
D. It allocates size 100 by default

Ans: B

Explanation: The compiler computes the exact size needed, which is the string's length plus one for '\0'.

Q15 | STATEMENT-BASED | CHARACTER STRINGS STR

Statement: "Every valid C string must be terminated with a null character." This statement is:

A. False
B. True only for globally declared arrays
C. True
D. True only for string literals

Ans: C

Explanation: Null termination is what makes a character array recognizable as a valid C string to library functions.

Q16 | FILL-IN-THE-BLANK | CHARACTER STRINGS STR

A string is essentially an array of ___ ending with a null character.

A. characters
B. pointers
C. integers
D. floats

Ans: A

Explanation: Strings are arrays of type char, terminated by '\0'.

Q17 | FUNCTION-BASED | CHARACTER STRINGS STR

Which of the following correctly displays a string using printf?

A. printf("%c", str);
B. printf("%f", str);
C. printf("%s", str);
D. printf("%d", str);

Ans: C

Explanation: The %s specifier tells printf to print characters until it reaches the null terminator.

Q18 | ASSERTION-REASON | CHARACTER STRINGS STR

Assertion (A): Strings are not a distinct built-in data type in C.
Reason (R): Strings are implemented using arrays of characters terminated by '\0'.

A. A is true, R is false
B. A is false, R is true
C. Both A and R are true, but R does not explain A
D. Both A and R are true, and R correctly explains A

Ans: D

Explanation: C has no dedicated 'string' type; strings are simply null-terminated character arrays, which is exactly why A is true.

Q19 | CONCEPTUAL | CHARACTER STRINGS STR

Which of the following statements about string storage is TRUE?

A. Characters of a string are stored in contiguous memory locations
B. Strings are stored on a separate stack
C. Strings cannot be stored in arrays
D. Strings are stored in reverse order in memory

Ans: A

Explanation: C strings occupy consecutive memory addresses, one byte per character, ending with '\0'.

Q20 | DIFFICULT | CHARACTER STRINGS STR

Given char str[20] = "OAVS"; what is guaranteed about indices 4 through 19?

A. They cause a compilation error
B. They contain unpredictable garbage values
C. They are all automatically initialized to '\0' (zero)
D. They contain repeated copies of "OAVS"

Ans: C

Explanation: When an initializer list is shorter than the array size, C guarantees the remaining elements are zero-initialized.

Q21 | DIFFICULT | CHARACTER STRINGS STR

Which of the following is TRUE regarding character arrays used to store strings?

A. The array must have at least length + 1 elements to accommodate the null terminator

B. A null terminator is optional in every case

C. Character arrays cannot store strings

D. The array size must exactly match the string's length

Ans: A

Explanation: An extra element is always required so that '\0' can be stored after the visible characters.

Q22 | CONCEPTUAL | CHARACTER STRINGS STR

What is the key difference between the string constant "A" and the character constant 'A'?

A. 'A' is a string; "A" is a character

B. There is no difference

C. "A" is a 2-element char array {'A','\0'}; 'A' is a single character

D. Both are treated as integers only

Ans: C

Explanation: Double quotes create a string (with an implicit null terminator); single quotes create one character.

Q23 | TRICKY | CHARACTER STRINGS STR

How many bytes does the empty string literal "" occupy in memory?

A. 2 bytes

B. Undefined, compiler dependent

C. 1 byte (just the null character)

D. 0 bytes

Ans: C

Explanation: Even an empty string needs storage for its single null terminator.

Q24 | APPLICATION-ORIENTED | CHARACTER STRINGS STR

Which loop condition is typically used to traverse a string character by character until its end?

A. while(i < 0)

B. while(str[i] != '\0')

C. while(str[i] == 1)

D. while(str[i] > 100)

Ans: B

Explanation: Traversal continues until the null terminator is encountered, since that marks the string's end.

Q25 | SCENARIO-BASED | CHARACTER STRINGS STR

A student writes char name[5] = "Rohit"; What is the likely consequence?

A. It automatically truncates to "Rohi"

B. It always compiles and runs with no risk

C. The compiler silently extends the array to size 6

D. "Rohit" needs 6 bytes (5 letters + '\0'); with no room left, the array is not null-terminated, risking undefined behavior

Ans: D

Explanation: Since the array size exactly equals the letter count, there is no space reserved for the terminator.

Q26 | FUNCTION-BASED | CHARACTER STRINGS STR

Which header file must be included to use standard string handling functions in C?

A. <string.h>

B. <stdio.h>

C. <stdlib.h>

D. <ctype.h>

Ans: A

Explanation: <string.h> declares strlen(), strcpy(), strcat(), strcmp(), and related functions.

Q27 | APPLICATION-ORIENTED | CHARACTER STRINGS STR

Strings are widely used in which of the following real applications?

A. Only boolean logic operations

B. Exclusively low-level graphics rendering

C. Only arithmetic calculations

D. Storing names, addresses, and other text-based data

Ans: D

Explanation: Text-based data such as names and addresses is naturally represented using strings.

Q28 | PREVIOUS-YEAR PATTERN | CHARACTER STRINGS STR

Which of the following correctly reads a full line of text into a string, including embedded spaces?

A. scanf("%s", str);

B. fgets(str, size, stdin);

C. scanf("%d", str);

D. getchar(str);

Ans: B

Explanation: fgets() reads an entire line (up to a size limit), including spaces, unlike scanf("%s").

Q29 | MATCH-THE-FOLLOWING | CHARACTER STRINGS STR

Match: (i) '\0' (ii) %s (iii) string.h (iv) char array with (1) format specifier (2) header file (3) null terminator (4) storage for a string

A. i-2, ii-1, iii-4, iv-3

B. i-4, ii-3, iii-2, iv-1

C. i-3, ii-1, iii-2, iv-4

D. i-1, ii-2, iii-3, iv-4

Ans: C

Explanation: '\0' terminates a string, %s formats it, string.h declares functions, and a char array stores it.

Q30 | COMPARISON-BASED | CHARACTER STRINGS STR

Which of the following is NOT true when comparing strings to a simple data type like int?

A. Strings need a terminating character; int does not

B. A string is stored as a single value in one memory cell, exactly like int

C. A string's storage size depends on its content length plus one

D. Strings are stored as arrays; int occupies a single memory location

Ans: B

Explanation: Unlike int, a string spans multiple contiguous memory cells (an array), not a single cell.

Q31 | TRICKY | CHARACTER STRINGS STR

Which of the following string initializations is INVALID (a compile-time error)?

A. char *str = "Hello";

B. char str[] = "Hello";

C. char str[5] = {'H','e','l','l','o','\0'};

D. char str[6] = {'H','e','l','l','o','\0'};

Ans: C

Explanation: An array of size 5 cannot hold 6 initializer values; this causes a 'too many initializers' error.

Q32 | TRICKY | CHARACTER STRINGS STR

Which of these correctly checks whether array 'str' currently holds an empty (zero-length) string?

A. if(str = "")

B. if(strlen(str) == -1)

C. if(str == 0)

D. if(str[0] == '\0')

Ans: D

Explanation: An empty string has its very first element equal to the null terminator.

Q33 | CONCEPTUAL | CHARACTER STRINGS **STR**

Displaying a string with printf("%s", str) primarily relies on:

A. The null character marking where output should stop

B. The size of the compiler being used

C. The system clock speed

D. The number of vowels in the string

Ans: A

Explanation: printf keeps printing characters until it reaches '\0', regardless of the array's allocated size.

Q34 | FILL-IN-THE-BLANK | CHARACTER STRINGS **STR**

In C, standard string handling functions are declared in the ___ header file.

A. string.h

B. stdio.h

C. conio.h

D. math.h

Ans: A

Explanation: string.h is the standard header for strlen(), strcpy(), strcat(), strcmp(), etc.

Q35 | CONCEPTUAL | CHARACTER STRINGS **STR**

Which option best summarizes the characteristics of strings in C?

A. Contents that can never be modified once created

B. A structure that is automatically resized at runtime

C. A fixed data type with automatic bounds checking

D. An array of characters, null-terminated, size fixed at declaration, contents mutable (unless a literal)

Ans: D

Explanation: Strings in C are simply null-terminated char arrays with a size fixed when declared.

Chapter 2: The strlen() Function

20 Questions | OAVS TGT Computer Science – Character Strings

Q36 | PURPOSE | CHARACTER STRINGS **LEN**

What does the strlen() function do?

A. Concatenates two strings

B. Copies a string

C. Compares two strings

D. Returns the length of a string excluding the null character

Ans: D

Explanation: strlen() counts characters in a string up to, but not including, the terminating '\0'.

Q37 | FUNCTION-BASED | CHARACTER STRINGS **LEN**

Which header file must be included to use strlen()?

A. <stdlib.h>

B. <stdio.h>

C. <string.h>

D. <ctype.h>

Ans: C

Explanation: strlen() is declared in <string.h>.

Q38 | RETURN VALUE | CHARACTER STRINGS **LEN**

What is the return type of strlen()?

A. float

B. int

C. char*

D. size_t (an unsigned integer type)

Ans: D

Explanation: The C standard defines strlen()'s return type as size_t, an unsigned integer type.

Q39 | SYNTAX | CHARACTER STRINGS **LEN**

Which is the correct usage syntax for strlen()?

A. strlen();

B. size_t len = strlen(str);

C. strlen(str1, str2);

D. len = strlen;

Ans: B

Explanation: strlen() takes exactly one string argument and returns its length, typically stored in a size_t variable.

Q40 | OUTPUT-BASED | CHARACTER STRINGS LEN

What is the output of: printf("%d", strlen("OAVS"));

A. 4
B. 0
C. 3
D. 5

Ans: A

Explanation: "OAVS" has 4 characters; strlen() excludes the null terminator from the count.

Q41 | PARAMETERS | CHARACTER STRINGS LEN

How many parameters does strlen() take?

A. 2
B. 3
C. 1
D. 0

Ans: C

Explanation: strlen() accepts a single pointer/array argument representing the string to be measured.

Q42 | OUTPUT-BASED | CHARACTER STRINGS LEN

What is the output of strlen("")?

A. 1
B. -1
C. 0
D. Undefined

Ans: C

Explanation: An empty string has zero characters before its null terminator.

Q43 | BEHAVIOR | CHARACTER STRINGS LEN

Does strlen() count the null character as part of its returned length?

A. Only for empty strings
B. No, it excludes the null character
C. Yes, always
D. Only for arrays larger than 10

Ans: B

Explanation: strlen() reports only the visible characters, never the terminator itself.

Q44 | CODE-SNIPPET | CHARACTER STRINGS LEN

char s[] = "Computer"; printf("%d", strlen(s)); What is the output?

A. 9
B. 7
C. 10
D. 8

Ans: D

Explanation: "Computer" contains 8 letters, so strlen() returns 8.

Q45 | LIMITATIONS | CHARACTER STRINGS LEN

Which situation could cause strlen() to behave unpredictably (undefined behavior)?

A. Passing a properly null-terminated string

B. Passing a character array that is NOT null-terminated

C. Passing an empty string

D. Passing a string literal

Ans: B

Explanation: Without a '\0' inside its bounds, strlen() keeps reading past the array, causing undefined behavior.

Q46 | APPLICATIONS | CHARACTER STRINGS LEN

What is a common application of strlen()?

A. Determining loop bounds while processing a string

B. Opening files

C. Sorting integers

D. Performing arithmetic addition

Ans: A

Explanation: Knowing a string's length is essential for safely looping through or validating its content.

Q47 | DIFFERENCES | CHARACTER STRINGS LEN

How does strlen() differ from sizeof() when applied to a character array?

A. They are fully interchangeable in every situation

B. strlen() returns the actual string length (excluding '\0'); sizeof() returns the total allocated array size

C. sizeof() excludes the null character; strlen() includes it

D. Both always return the same value

Ans: B

Explanation: sizeof() reports the compile-time allocated size, while strlen() reports the runtime content length.

Q48 | ASSERTION-REASON | CHARACTER STRINGS LEN

Assertion (A): strlen() can be safely used on any character array regardless of null-termination.
Reason (R): strlen() keeps counting characters until it encounters a '\0'.

A. Both A and R are false

B. A is false, but R correctly describes how strlen() works

C. Both A and R are true, and R explains A

D. A is true, R is false

Ans: B

Explanation: strlen() is unsafe on non-terminated arrays precisely because it relies on finding '\0', which R correctly states.

Q49 | TRICKY | CHARACTER STRINGS LEN

What is the outcome of calling strlen(NULL)?

A. It returns -1
B. Undefined behavior (commonly a crash)
C. It returns 0
D. It returns 1

Ans: B

Explanation: Passing a NULL pointer to strlen() is undefined behavior since there is no memory to scan.

Q50 | OUTPUT-BASED | CHARACTER STRINGS LEN

char a[20] = "Hi"; printf("%d", strlen(a)); What is the output?

A. 18
B. 2
C. 3
D. 20

Ans: B

Explanation: strlen() only counts the actual characters before '\0' ("Hi" = 2), ignoring unused allocated space.

Q51 | CONCEPTUAL | CHARACTER STRINGS LEN

Which manual loop logic mirrors what strlen() internally performs?

A. while(1) break;
B. for(i=0; str[i]!='\0'; i++) count++;
C. for(i=0;i<10;i++) sum+=i;
D. for(i=n;i>0;i--) count--;

Ans: B

Explanation: strlen() effectively scans forward, incrementing a counter until it hits the null terminator.

Q52 | SCENARIO-BASED | CHARACTER STRINGS LEN

A program must verify a password string is at least 8 characters long. Which function directly helps?

A. strlen()
B. strcpy()
C. strcat()
D. strcmp()

Ans: A

Explanation: strlen() reports the exact character count needed for this length check.

Q53 | APPLICATION-ORIENTED | CHARACTER STRINGS **LEN**

Which format specifier correctly matches strlen()'s return type for printing?

A. %p only
B. %c
C. %zu (matches size_t)
D. %f

Ans: C

Explanation: %zu is the standard format specifier for size_t values, though %d is often used informally for small lengths.

Q54 | PREVIOUS-YEAR PATTERN | CHARACTER STRINGS **LEN**

Which statement about strlen() is FALSE?

A. It does not modify the original string
B. It computes length by scanning until '\0'
C. It returns a signed negative number for invalid input
D. It requires <string.h>

Ans: C

Explanation: strlen()'s return type (size_t) is unsigned, so it never returns a negative value.

Q55 | COMPARISON-BASED | CHARACTER STRINGS **LEN**

Unlike strcmp(), the strlen() function:

A. compares two strings against each other
B. always returns either 0 or 1
C. copies string contents into another array
D. operates on a single string argument only

Ans: D

Explanation: strlen() takes just one string, whereas strcmp() requires two strings to compare.

Chapter 3: The strcat() Function

20 Questions | OAVS TGT Computer Science – Character Strings

Q56 | PURPOSE | CHARACTER STRINGS CAT

What is the purpose of the strcat() function?

A. Appends (concatenates) one string to the end of another

B. Finds the length of a string

C. Copies one string to another

D. Compares two strings

Ans: A

Explanation: strcat() joins the source string onto the end of the destination string.

Q57 | FUNCTION-BASED | CHARACTER STRINGS CAT

Which header file declares strcat()?

A. <stdlib.h>

B. <string.h>

C. <math.h>

D. <stdio.h>

Ans: B

Explanation: strcat() is part of the standard string library declared in <string.h>.

Q58 | RETURN VALUE | CHARACTER STRINGS CAT

What does strcat(dest, src) return?

A. void (nothing)

B. An int representing status

C. A pointer to src

D. A pointer to dest

Ans: D

Explanation: strcat() returns the same pointer passed in as dest, now containing the combined string.

Q59 | CODE-SNIPPET | CHARACTER STRINGS CAT

char a[20]="Hello "; char b[]="World"; strcat(a,b); printf("%s",a); What is the output?

A. HelloWorld

B. Hello

C. Hello World

D. World Hello

Ans: C

Explanation: strcat() appends "World" right after "Hello " (which already has a trailing space).

Q60 | BEHAVIOR | CHARACTER STRINGS **CAT**

What must be true about the destination array for strcat() to work safely?

A. It must have enough space to hold both strings plus the null terminator

B. It must start out completely empty

C. It must be declared with the const keyword

D. It must be exactly the size of src

Ans: A

Explanation: Insufficient space in dest leads to buffer overflow, since strcat() performs no bounds checking.

Q61 | BEHAVIOR | CHARACTER STRINGS **CAT**

Where does strcat() begin writing the appended characters into dest?

A. Right after src's own null terminator

B. At a randomly chosen position

C. At the position of dest's existing null terminator

D. At the very beginning, overwriting dest entirely

Ans: C

Explanation: strcat() locates dest's terminating '\0' first, then starts copying src's characters from there.

Q62 | BEHAVIOR | CHARACTER STRINGS **CAT**

What happens to dest's original null terminator during a strcat() call?

A. It is overwritten, and a new null terminator is placed at the very end

B. It is removed permanently with nothing replacing it

C. It gets duplicated twice

D. It stays in the middle of the final combined string

Ans: A

Explanation: The old terminator is overwritten by the first character of src; a fresh '\0' ends the final result.

Q63 | TRICKY | CHARACTER STRINGS **CAT**

What can happen if strcat() is used without ensuring the destination array is large enough?

A. A guaranteed compilation error

B. A buffer overflow / undefined behavior

C. Nothing; C automatically resizes arrays at runtime

D. The extra characters are safely and silently discarded

Ans: B

Explanation: strcat() blindly writes past the array bounds if there isn't enough room, corrupting memory.

Q64 | STATEMENT-BASED | CHARACTER STRINGS **CAT**

Which of the following is a valid use of strcat()?

A. strcat("Hello", "World");
B. char dest[20]="Hello"; strcat(dest,"World");
C. strcat();
D. strcat(5, "World");

Ans: B

Explanation: dest must be a writable character array; string literals (as in option A) are typically not modifiable.

Q65 | OUTPUT-BASED | CHARACTER STRINGS **CAT**

char s[10]="Hi"; strcat(s,"!"); printf("%s",s); What is the output?

A. Hi
B. Hi!
C. !Hi
D. Compilation error

Ans: B

Explanation: strcat() appends "!" right after the existing content "Hi".

Q66 | APPLICATIONS | CHARACTER STRINGS **CAT**

strcat() is commonly used to:

A. Convert a string into an integer
B. Merge/build up strings, such as combining a directory path with a filename
C. Sort an array of characters
D. Delete a portion of a string

Ans: B

Explanation: Building composite strings from smaller pieces is a classic use case for strcat().

Q67 | DIFFERENCES | CHARACTER STRINGS **CAT**

What is the key difference between strcat() and strcpy()?

A. strcat() only works with integer arrays
B. strcat() overwrites the destination; strcpy() appends to it
C. Both functions behave identically
D. strcat() appends src to dest; strcpy() overwrites dest with src

Ans: D

Explanation: strcpy() replaces dest's content, while strcat() adds src onto the end of dest's existing content.

Q68 | ASSERTION-REASON | CHARACTER STRINGS **CAT**

Assertion (A): strcat(dest, src) can cause undefined behavior if dest and src overlap in memory.
Reason (R): strcat() performs no check for overlapping memory regions.

A. Both A and R are false
B. A is true, R is false
C. Both A and R are true, and R correctly explains A
D. A is false, R is true

Ans: C

Explanation: Overlapping source and destination is unsafe precisely because strcat() does not guard against it.

Q69 | CONCEPTUAL | CHARACTER STRINGS **CAT**

What is the practical use of strcat()'s return value?

A. It allows chaining, e.g. passing the result directly into another function call, since it returns dest's pointer
B. It is mainly used for error checking
C. It represents the new combined string's length
D. It is always ignored by the compiler

Ans: A

Explanation: Because strcat() returns dest, its result can be passed straight into functions like printf().

Q70 | PARAMETERS | CHARACTER STRINGS **CAT**

Which statement about strcat()'s parameters is TRUE?

A. Both parameters must be integers
B. Neither parameter needs a null terminator
C. Both dest and src must be properly null-terminated strings
D. Only the second parameter needs a null terminator

Ans: C

Explanation: strcat() relies on null terminators in both dest (to find where to start) and src (to know where to stop).

Q71 | SCENARIO-BASED | CHARACTER STRINGS **CAT**

To join a first name and last name with a space between them using strcat(), which sequence works correctly?

A. strcat(first,"\0"); strcat(first,last);
B. strcat(first,last," ");
C. strcat(first," "); strcat(first,last);
D. strcat(last,first);

Ans: C

Explanation: First appending a space, then the last name, correctly builds "First Last" in the first array.

Q72 | FILL-IN-THE-BLANK | CHARACTER STRINGS **CAT**

strcat() appends a copy of the source string, ____ its terminating null character, onto the destination string.

A. including
B. duplicating
C. excluding
D. reversing

Ans: A

Explanation: The appended copy of src brings its own '\0' along, which becomes the new end of dest.

Q73 | PREVIOUS-YEAR PATTERN | CHARACTER STRINGS **CAT**

What is a key limitation of strcat()?

A. It requires both strings to be identical
B. It cannot handle strings longer than 5 characters
C. It cannot be used with character arrays at all
D. It performs no bounds checking, risking buffer overflow if dest is too small

Ans: D

Explanation: Like strcpy(), strcat() trusts the programmer to ensure dest has sufficient space.

Q74 | OUTPUT-BASED | CHARACTER STRINGS **CAT**

char x[15] = "OA"; strcat(x, "VS"); strcat(x, " Odisha"); printf("%s", x); What is the output?

A. VSOA Odisha
B. OAVS Odisha
C. Odisha OAVS
D. OA VS Odisha

Ans: B

Explanation: Successive strcat() calls keep appending onto the growing string in x.

Q75 | CONCEPTUAL | CHARACTER STRINGS **CAT**

Which manual process is functionally similar to what strcat() automates?

A. Reversing the characters of a string
B. Finding dest's null terminator, then copying src character-by-character from that point, ending with a new '\0'
C. Converting lowercase letters to uppercase
D. Comparing each character of two strings for equality

Ans: B

Explanation: This describes exactly what strcat() does internally when joining two strings.

Chapter 4: The strcpy() Function

20 Questions | OAVS TGT Computer Science – Character Strings

Q76 | PURPOSE | CHARACTER STRINGS CPY

What is the purpose of the strcpy() function?

A. Compares two strings

B. Concatenates two strings

C. Copies the contents of one string into another

D. Finds the length of a string

Ans: C

Explanation: strcpy() copies characters from a source string into a destination array.

Q77 | FUNCTION-BASED | CHARACTER STRINGS CPY

Which header file declares strcpy()?

A. <stdio.h>

B. <string.h>

C. <stdlib.h>

D. <ctype.h>

Ans: B

Explanation: strcpy() is declared in the standard <string.h> header.

Q78 | SYNTAX | CHARACTER STRINGS CPY

In char *strcpy(char *dest, const char *src);, what does the 'const' before src indicate?

A. src can still be modified by strcpy()

B. src is only read by strcpy() and is never modified

C. src must be a constant integer value

D. dest cannot be changed by the function

Ans: B

Explanation: 'const' signals that strcpy() only reads from src; it never alters the source string.

Q79 | RETURN VALUE | CHARACTER STRINGS CPY

What does strcpy() return?

A. The length of the copied string

B. A pointer to the source string

C. 0 or 1 to indicate success/failure

D. A pointer to the destination string

Ans: D

Explanation: strcpy() returns the same pointer passed as dest, now holding a copy of src's contents.

Q80 | CODE-SNIPPET | CHARACTER STRINGS CPY

char dest[20]; strcpy(dest,"OAVS"); printf("%s",dest); What is the output?

A. (empty output)
B. OAVS
C. Compilation error
D. dest

Ans: B

Explanation: strcpy() copies "OAVS" (including its terminator) into dest, so printf displays "OAVS".

Q81 | BEHAVIOR | CHARACTER STRINGS CPY

Besides the visible characters, what else does strcpy() copy from src into dest?

A. Nothing extra beyond the letters
B. The terminating null character '\0'
C. Random garbage bytes
D. Only the first character

Ans: B

Explanation: strcpy() copies characters up to and including src's null terminator.

Q82 | BEHAVIOR | CHARACTER STRINGS CPY

What is required of the destination array for strcpy() to work safely?

A. It must be at least as large as src, including room for the null terminator
B. It must be a string literal
C. It must be smaller than the source string
D. It must be declared with const

Ans: A

Explanation: If dest is too small, strcpy() overflows it since it performs no bounds checking.

Q83 | TRICKY | CHARACTER STRINGS CPY

What happens if strcpy() is used to write into a string literal, e.g. strcpy(literalPtr, "xyz") where literalPtr points to a literal?

A. It swaps the two strings
B. It works fine and modifies the literal
C. Undefined behavior, since string literals are typically stored in non-modifiable memory
D. It always produces a compilation error

Ans: C

Explanation: Attempting to modify a string literal's storage is undefined behavior in C.

Q84 | DIFFERENCES | CHARACTER STRINGS **CPY**

What is the key difference between strcpy() and strcat()?

A. strcpy() only works on integer arrays

B. strcpy() appends; strcat() overwrites

C. Both perform the exact same operation

D. strcpy() overwrites dest; strcat() appends onto the end of dest

Ans: D

Explanation: strcpy() replaces dest's entire content, whereas strcat() adds onto what's already there.

Q85 | OUTPUT-BASED | CHARACTER STRINGS **CPY**

char a[10]="Hello"; char b[10]; strcpy(b,a); printf("%s",b); What is the output?

A. b

B. Compilation error

C. (empty output)

D. Hello

Ans: D

Explanation: strcpy() duplicates the contents of a into b, so b now also holds "Hello".

Q86 | APPLICATIONS | CHARACTER STRINGS **CPY**

strcpy() is typically used to:

A. Reverse an array of integers

B. Compare two numeric values for equality

C. Perform mathematical rounding

D. Initialize or reset a character array with new string content at runtime

Ans: D

Explanation: Assigning fresh text into an existing char array at runtime is the classic use of strcpy().

Q87 | LIMITATIONS | CHARACTER STRINGS **CPY**

What is a key limitation of strcpy()?

A. It requires both arrays to be exactly the same size

B. It automatically checks buffer sizes for safety

C. It cannot copy strings longer than 5 characters

D. It does not check whether dest has enough space, risking buffer overflow

Ans: D

Explanation: Like strcat(), strcpy() offers no built-in protection against overflowing a too-small destination.

Q88 | ASSERTION-REASON | CHARACTER STRINGS **CPY**

Assertion (A): strcpy(dest, src) is unsafe when dest is smaller than src.
Reason (R): strcpy() copies characters until it reaches src's null terminator, without checking dest's boundary.

A. A is true, R is false
B. A is false, R is true
C. Both A and R are false
D. Both A and R are true, and R correctly explains A

Ans: D

Explanation: This is exactly why a too-small dest causes a buffer overflow when copying a longer src.

Q89 | CONCEPTUAL | CHARACTER STRINGS **CPY**

Which related (though non-syllabus) function is sometimes used as a safer alternative to strcpy() by limiting the number of characters copied?

A. strlen()
B. strncpy()
C. strcat()
D. strcmp()

Ans: B

Explanation: strncpy() adds a maximum-character-count parameter, reducing (but not eliminating) overflow risk.

Q90 | SCENARIO-BASED | CHARACTER STRINGS **CPY**

A program needs to duplicate a string in 'source' into a new array 'backup' for safekeeping. Which function call is most direct?

A. strlen(source);
B. strcat(source, backup);
C. strcmp(source, backup);
D. strcpy(backup, source);

Ans: D

Explanation: strcpy() copies source's content into backup, exactly matching the intended duplication.

Q91 | FILL-IN-THE-BLANK | CHARACTER STRINGS **CPY**

strcpy() copies characters from src to dest ___ the terminating null character has also been copied.

A. before
B. without
C. until
D. after

Ans: C

Explanation: Copying continues, character by character, until '\0' itself has been transferred.

Q92 | OUTPUT-BASED | CHARACTER STRINGS CPY

char s[6]; strcpy(s,"OAVS"); printf("%d", strlen(s)); What is the output?

A. 6
B. 0
C. 4
D. 5

Ans: C

Explanation: strlen() reports the 4 visible characters of "OAVS", not the array's allocated size.

Q93 | PREVIOUS-YEAR PATTERN | CHARACTER STRINGS CPY

Which statement is TRUE regarding strcpy()'s two parameters?

A. Only src is modified; dest remains unchanged
B. Both parameters are modified after the call
C. Neither parameter is changed by the call
D. Only dest is modified; src remains unchanged

Ans: D

Explanation: strcpy() writes into dest only; src is treated as read-only (as reflected by its 'const' qualifier).

Q94 | DIFFICULT | CHARACTER STRINGS CPY

Why is calling strcpy(dest, dest) considered redundant and conceptually risky?

A. Source and destination fully overlap here; in general, overlapping regions passed to strcpy() are undefined behavior
B. It automatically clears the contents of dest
C. It always crashes the compiler immediately
D. It silently swaps the characters of dest

Ans: A

Explanation: Even though copying onto itself seems harmless, overlapping arguments are technically undefined behavior for strcpy().

Q95 | COMPARISON-BASED | CHARACTER STRINGS CPY

Unlike strcmp(), which only reads its two string arguments, strcpy():

A. writes into (modifies) its destination argument
B. never accesses memory at all
C. also only reads both of its arguments
D. returns a true/false boolean value

Ans: A

Explanation: strcpy() actively modifies dest by writing src's characters into it.

Chapter 5: The strcmp() Function

20 Questions | OAVS TGT Computer Science – Character Strings

Q96 | PURPOSE | CHARACTER STRINGS **CMP**

What is the purpose of the strcmp() function?

A. Finds the length of a string
B. Copies one string into another
C. Compares two strings lexicographically
D. Concatenates two strings

Ans: C

Explanation: strcmp() compares two strings character by character based on their ASCII values.

Q97 | FUNCTION-BASED | CHARACTER STRINGS **CMP**

Which header file declares strcmp()?

A. <stdio.h>
B. <stdlib.h>
C. <ctype.h>
D. <string.h>

Ans: D

Explanation: strcmp() is part of the standard string library in <string.h>.

Q98 | RETURN VALUE | CHARACTER STRINGS **CMP**

What does strcmp() return when both strings are exactly equal?

A. -1
B. 0
C. NULL
D. 1

Ans: B

Explanation: A return value of 0 indicates that both strings match exactly.

Q99 | RETURN VALUE | CHARACTER STRINGS **CMP**

What does strcmp() return when the first string is lexicographically greater than the second?

A. The string itself
B. Zero
C. A negative value
D. A positive value

Ans: D

Explanation: A positive return value means the first string comes after the second in lexicographic order.

Q100 | RETURN VALUE | CHARACTER STRINGS **CMP**

What does strcmp() return when the first string is lexicographically smaller than the second?

A. Zero
B. A positive value
C. Undefined
D. A negative value

Ans: D

Explanation: A negative return value means the first string comes before the second in lexicographic order.

Q101 | SYNTAX | CHARACTER STRINGS **CMP**

In int strcmp(const char *s1, const char *s2);, what is the return type?

A. void
B. int
C. char
D. float

Ans: B

Explanation: strcmp() returns an int whose sign indicates the comparison result.

Q102 | CODE-SNIPPET | CHARACTER STRINGS **CMP**

if(strcmp("apple","apple")==0) printf("Equal"); What is the output?

A. Not Equal
B. Compilation error
C. No output
D. Equal

Ans: D

Explanation: Since both strings are identical, strcmp() returns 0, so the condition is true.

Q103 | BEHAVIOR | CHARACTER STRINGS **CMP**

On what basis does strcmp() compare characters?

A. Alphabetical position, ignoring case, only
B. Only the total length of each string
C. Their memory addresses
D. Their ASCII (numeric) values, character by character

Ans: D

Explanation: strcmp() compares corresponding characters' ASCII codes until a mismatch or a null terminator is found.

Q104 | OUTPUT-BASED | CHARACTER STRINGS **CMP**

What can we conclude from: strcmp("apple","banana") returning a negative value?

A. The two strings are equal

B. 'a' has a smaller ASCII value than 'b', so "apple" precedes "banana"

C. This indicates a runtime error

D. "apple" is longer than "banana"

Ans: B

Explanation: The first differing characters are 'a' and 'b'; since 'a' < 'b' in ASCII, the result is negative.

Q105 | APPLICATIONS | CHARACTER STRINGS **CMP**

What is a practical use of strcmp()?

A. Checking user input against a stored password string

B. Reversing the characters of a string

C. Adding two strings together numerically

D. Measuring the length of a string

Ans: A

Explanation: strcmp() is ideal for verifying exact matches, such as password or command validation.

Q106 | TRICKY | CHARACTER STRINGS **CMP**

Does strcmp() perform a case-sensitive comparison?

A. This depends entirely on the compiler used

B. Only the first letter's case matters

C. Yes, "Apple" and "apple" are treated as different strings

D. No, it always ignores case

Ans: C

Explanation: Because comparison is based on raw ASCII values, uppercase and lowercase letters differ.

Q107 | CONCEPTUAL | CHARACTER STRINGS **CMP**

Which of these is used (though non-standard/compiler-specific) for case-insensitive string comparison, unlike strcmp()?

A. A function like strcmpi()/stricmp()

B. strlen()

C. strcmp()

D. strcpy()

Ans: A

Explanation: strcmp() itself is always case-sensitive; case-insensitive comparison needs a different, non-ANSI-standard function.

Q108 | ASSERTION-REASON | CHARACTER STRINGS **CMP**

Assertion (A): strcmp() alone cannot be used as the basis of a sorting comparison function for strings.
Reason (R): strcmp() only returns 0, a negative value, or a positive value, which is sufficient information to order two strings.

A. A is true, R is false
B. Both A and R are false
C. Both A and R are true, and R explains A
D. A is false, and R correctly explains why strcmp() actually works well for sorting

Ans: D

Explanation: strcmp()'s three possible outcomes (0, negative, positive) are exactly what sorting comparators require, making A false.

Q109 | OUTPUT-BASED | CHARACTER STRINGS **CMP**

What is the sign of strcmp("Zebra","apple")?

A. Negative, because uppercase 'Z' (90) has a lower ASCII value than lowercase 'a' (97)
B. Cannot be determined without running the code
C. Positive
D. Zero, since both start with letters

Ans: A

Explanation: Uppercase letters have lower ASCII values than lowercase letters, so "Zebra" compares as 'less than' "apple".

Q110 | SCENARIO-BASED | CHARACTER STRINGS **CMP**

A menu-driven program checks whether the user typed "exit" to quit. Which function directly helps?

A. strlen()
B. strcmp()
C. strcat()
D. strcpy()

Ans: B

Explanation: strcmp() confirms an exact match between the user's input and the string "exit".

Q111 | FILL-IN-THE-BLANK | CHARACTER STRINGS **CMP**

strcmp() stops comparing further characters as soon as it finds ___ characters, or reaches the end of either string.

A. identical
B. uppercase
C. numeric
D. differing

Ans: D

Explanation: Comparison halts at the first point of difference, since the outcome is already determined.

Q112 | PREVIOUS-YEAR PATTERN | CHARACTER STRINGS **CMP**

What does the C standard actually guarantee about strcmp()'s numeric return value when two strings differ?

A. It always equals the exact ASCII difference of the differing characters

B. It is always exactly +1 or -1

C. Only the sign is guaranteed to indicate order; the exact magnitude is implementation-defined

D. It is always exactly +100 or -100

Ans: C

Explanation: Programmers should test the sign of the result (e.g., <0, ==0, >0), not rely on a specific numeric value.

Q113 | COMPARISON-BASED | CHARACTER STRINGS **CMP**

How does strcmp() differ from directly using == on two character arrays (str1 == str2)?

A. Both perform identical content-based comparison

B. == always evaluates to true for equal-content strings

C. strcmp() compares addresses only, not content

D. strcmp() compares actual string content; == on arrays compares their addresses, not content

Ans: D

Explanation: Arrays decay to pointers in such an expression, so == compares memory addresses, not the text inside them.

Q114 | APPLICATIONS | CHARACTER STRINGS **CMP**

strcmp() plays an essential role in implementing:

A. General file compression algorithms

B. Mathematical square root calculations

C. Sorting algorithms for arrays of strings and searching for exact matches

D. Random number generation

Ans: C

Explanation: Both alphabetical sorting and exact-match searching of strings rely on strcmp()'s comparison result.

Q115 | COMPARISON-BASED | CHARACTER STRINGS **CMP**

While strlen() takes exactly one argument, strcmp() takes:

A. A variable number of arguments

B. Zero arguments

C. Three arguments

D. Two arguments (the two strings being compared)

Ans: D

Explanation: strcmp() needs two strings, s1 and s2, in order to compare them against each other.

Chapter 6: Strings vs One-Dimensional Character Arrays

25 Questions | OAVS TGT Computer Science – Character Strings

Q116 | CONCEPTUAL | CHARACTER STRINGS **CMP2**

What is the fundamental relationship between a string and a one-dimensional character array in C?

A. Strings are a built-in type completely distinct from arrays

B. They are entirely unrelated concepts

C. A string is essentially implemented as a one-dimensional character array terminated by '\0'

D. A string is always a two-dimensional array

Ans: C

Explanation: C has no separate string type; strings ARE null-terminated one-dimensional char arrays.

Q117 | COMPARISON-BASED | CHARACTER STRINGS **CMP2**

Which of these is a genuine similarity between strings and general 1-D character arrays?

A. Both store elements of type char in contiguous memory

B. Both are automatically sorted by the compiler

C. Neither can be accessed using indexing

D. Both must always have exactly 10 elements

Ans: A

Explanation: Both are simply arrays of char stored in sequential memory locations.

Q118 | CONCEPTUAL | CHARACTER STRINGS **CMP2**

What key feature distinguishes a 'string' from a plain character array holding arbitrary (non-text) data?

A. A string must be terminated with '\0'; a general char array need not have this terminator or represent text

B. Strings cannot be indexed, but plain arrays can

C. A string may only contain digit characters

D. There is no distinction between the two in any respect

Ans: A

Explanation: The presence of a null terminator (marking valid text data) is what makes an array a proper string.

Q119 | ACCESS METHODS | CHARACTER STRINGS **CMP2**

How is an individual character within a string/character array accessed?

A. Individual characters cannot be accessed
B. Only by calling strcmp()
C. Using indexing syntax such as str[i]
D. Only by calling strcat()

Ans: C

Explanation: Array indexing (str[i]) directly accesses the character at position i.

Q120 | INTERNAL REPRESENTATION | CHARACTER STRINGS **CMP2**

Given char arr[5] = {'H','e','l','l','o'};, is 'arr' a valid, safely usable C-string?

A. Only when compiled using a C++ compiler
B. Yes, always
C. Yes, because it has exactly 5 characters
D. No — there is no null terminator within the array to mark where the text ends

Ans: D

Explanation: All 5 slots are filled with letters, leaving no room for a '\0', so functions like strlen() cannot safely process it.

Q121 | MEMORY ORGANIZATION | CHARACTER STRINGS **CMP2**

A string holding n visible characters typically requires how much storage (one byte per char)?

A. n + 1 bytes (n characters plus the null terminator)
B. 2n bytes
C. n - 1 bytes
D. n bytes

Ans: A

Explanation: The extra byte accounts for the mandatory terminating '\0'.

Q122 | NULL TERMINATION | CHARACTER STRINGS **CMP2**

Which of the following is TRUE regarding null termination?

A. Null termination applies only to arrays of integers
B. Null termination is optional even for valid strings
C. Every character array is automatically null-terminated by the compiler
D. Only arrays intended to represent strings need an explicit '\0'; a general-purpose char array doesn't require one

Ans: D

Explanation: Plain char arrays used for non-string data (like a fixed buffer of grades) have no such requirement.

Q123 | PRACTICAL IMPLICATIONS | CHARACTER STRINGS **CMP2**

Why is it unsafe to call strlen() on a character array that is not null-terminated?

A. strlen() simply refuses to compile in that case

B. strlen() automatically returns 0 in that case

C. strlen() keeps reading memory past the array's boundary searching for '\0', causing undefined behavior

D. strlen() returns the array's declared size instead of scanning it

Ans: C

Explanation: With no terminator inside the array, the scan runs into unrelated memory, which is undefined behavior.

Q124 | ACCESS METHODS | CHARACTER STRINGS **CMP2**

What is the standard way to access characters of a string / character array in C?

A. Through simple array indexing, e.g., str[i]

B. Only through file input/output operations

C. Only via the strcmp() function

D. Only through complex pointer arithmetic

Ans: A

Explanation: Indexing is the most direct and common way to access individual characters.

Q125 | ARRAY NOTATION | CHARACTER STRINGS **CMP2**

In char str[] = "OAVS";, what does str[0] refer to?

A. The first character 'O'

B. The entire string at once

C. The null terminator

D. The last character 'S'

Ans: A

Explanation: Array indices start at 0, so str[0] is the string's first character.

Q126 | COMPARISON-BASED | CHARACTER STRINGS **CMP2**

How does the total allocated size of a char array typically compare to the 'string length' it holds?

A. Allocated size can be greater than or equal to (string length + 1); strlen() only reports characters before '\0'

B. Allocated size is always smaller than the string's length

C. There is never any relationship between the two

D. They are always exactly identical

Ans: A

Explanation: An array can be declared larger than currently needed, while strlen() reports only the actual content length.

Q127 | TRICKY | CHARACTER STRINGS **CMP2**

For char arr[10] = "Hi";, what are the values of sizeof(arr) and strlen(arr) respectively?

A. sizeof = 2, strlen = 2

B. sizeof = 10, strlen = 2

C. sizeof = 2, strlen = 10

D. sizeof = 10, strlen = 10

Ans: B

Explanation: sizeof() reports the full declared array size (10 bytes); strlen() reports only "Hi"'s 2 characters.

Q128 | DIFFERENCES | CHARACTER STRINGS **CMP2**

Can a plain 1-D character array (not intended as text) be reliably used with strlen() or strcat()?

A. Never, under any circumstances whatsoever

B. Only if the array's size is less than 5

C. Yes, always, regardless of its contents

D. Only if it happens to contain a '\0' terminator somewhere within its declared bounds

Ans: D

Explanation: Without a null terminator present, these functions cannot correctly determine where the 'string' ends.

Q129 | SCENARIO-BASED | CHARACTER STRINGS **CMP2**

A programmer declares char grades[5]; and fills all 5 slots with letter grades, leaving no '\0'. Can this be safely passed to strlen()?

A. No — without a null terminator inside its bounds, strlen() causes undefined behavior by reading beyond the array

B. Only if grades is declared as a static array

C. Yes, strlen() will simply return 5 in this case

D. Yes, it is always completely safe

Ans: A

Explanation: strlen() will keep scanning past the array's end looking for a '\0' that was never placed there.

Q130 | INITIALIZATION | CHARACTER STRINGS **CMP2**

How does initializing a string literal differ from initializing a general character array with individual characters?

A. char s[] = "Hi"; automatically appends '\0'; char arr[2] = {'H','i'}; does NOT include a null terminator

B. Both automatically receive a null terminator

C. Neither automatically receives a null terminator

D. Only arrays declared as static automatically get a null terminator

Ans: A

Explanation: String-literal initialization always adds '\0'; manually listing characters equal to the array size does not.

Q131 | APPLICATION-ORIENTED | CHARACTER STRINGS **CMP2**

Why do C programmers rely on treating strings as char arrays with a guaranteed '\0' terminator?

A. It offers no practical benefit at all

B. It allows standard library functions (strlen, strcpy, strcat, strcmp, printf's %s) to correctly determine where the text ends

C. It prevents any form of array indexing on the data

D. It is required only for performing arithmetic operations

Ans: B

Explanation: The terminator is the shared convention that all string-handling functions depend on.

Q132 | STATEMENT-BASED | CHARACTER STRINGS **CMP2**

Statement: 'Every one-dimensional character array in C is automatically treated as a string by the compiler.' This is:

A. True only for arrays declared with the const keyword

B. True only when using a C++ compiler

C. True

D. False — an array only qualifies as a valid C-string if it is properly null-terminated

Ans: D

Explanation: The compiler does not enforce null-termination; it is the programmer's responsibility to ensure it for string usage.

Q133 | MATCH-THE-FOLLOWING | CHARACTER STRINGS CMP2

Match: (i) char str[6]="Hello"; (ii) char arr[5]={'H','e','l','l','o'}; (iii) strlen(str) (iv) str[5] with (1) 5 (2) '\0' (3) Valid null-terminated string (4) Not a valid string (no terminator)

A. i-3, ii-4, iii-1, iv-2

B. i-2, ii-1, iii-4, iv-3

C. i-4, ii-3, iii-2, iv-1

D. i-1, ii-2, iii-3, iv-4

Ans: A

Explanation: str is a valid string (i-3), arr lacks a terminator (ii-4), strlen(str)=5 (iii-1), and str[5] holds '\0' (iv-2).

Q134 | CHARACTER INDEXING | CHARACTER STRINGS CMP2

Which statement best explains 'character indexing' as it applies to strings?

A. Characters within a string cannot be individually accessed

B. Indexing of string characters starts from 1 in C

C. Each character occupies a unique position starting from index 0, accessible via str[index]

D. Only the very last character of a string can be indexed

Ans: C

Explanation: C uses zero-based indexing for all arrays, strings included.

Q135 | PRACTICAL IMPLICATIONS | CHARACTER STRINGS CMP2

What is a key practical implication of using a null terminator in strings?

A. It allows functions to determine a string's length dynamically without needing a separate length parameter

B. It effectively converts the array into a linked list

C. It prevents any function from ever reading the array

D. It doubles the memory required to store any string

Ans: A

Explanation: Because the terminator marks the end, functions like strlen() and printf() never need to be told the length explicitly.

Q136 | PREVIOUS-YEAR PATTERN | CHARACTER STRINGS CMP2

Which of the following declarations creates an array that IS a valid, properly null-terminated string?

A. char a[4] = {'C','A','T','\0'};

B. char a[3] = {'C','A','T'};

C. char a[3] = {'C','A'};

D. char a[2] = {'C','A','T'};

Ans: A

Explanation: Here, all 3 letters plus an explicit '\0' fit exactly within the declared size of 4.

Q137 | DIFFICULT | CHARACTER STRINGS **CMP2**

Given char arr[6] = "ABCDE"; and char brr[5] = {'A','B','C','D','E'};, which array is guaranteed to be a valid null-terminated string?

A. Only arr, since its extra 6th slot is automatically set to '\0'

B. Neither array is a valid string

C. Only brr

D. Both arrays are guaranteed valid strings

Ans: A

Explanation: arr has one spare byte reserved for the terminator; brr's 5 slots are entirely filled with letters, leaving no room for '\0'.

Q138 | MEMORY ORGANIZATION | CHARACTER STRINGS **CMP2**

Which statement accurately describes the memory organization of a string?

A. Characters of a string are stored in scattered, non-contiguous memory locations

B. Strings always require exactly double the memory of an equivalent plain array

C. Characters of a string occupy contiguous, sequential memory addresses, ending with '\0'

D. Strings are stored in reverse memory order compared to ordinary arrays

Ans: C

Explanation: Just like any array, a string's characters sit one after another in memory, followed by the terminator.

Q139 | TRICKY | CHARACTER STRINGS **CMP2**

Is char c = 'A'; equivalent to char str[] = "A";?

A. Yes, both are identical in memory representation

B. No — 'A' is a single character occupying 1 byte, while "A" is a 2-element array {'A','\0'}

C. No — 'A' actually requires more memory than "A"

D. Yes, because both ultimately represent the letter A

Ans: B

Explanation: A character constant is one byte; a string literal always includes an implicit null terminator, making it at least 2 bytes here.

Q140 | APPLICATION-ORIENTED | CHARACTER STRINGS **CMP2**

For exam purposes, what is the single most important factor distinguishing a string from a general character array?

A. Strings are restricted to holding only numeric digits

B. The array's size must always be an even number

C. The presence (in a string) or absence (in a general array) of a null terminator marking valid string data

D. Character arrays can never be printed using %s under any circumstance

Ans: C

Explanation: This null-terminator distinction is the core concept tested across OAVS-style questions on this topic.